

weatherisk: A Python Package for Spatial Clustering of Climate Extremes via Max-Stable Processes

Widad Kchikach^{a,b,*}, Thorsten Dickhaus^b, Gerrit Lohmann^{a,b}

^a*Alfred Wegener Institute Helmholtz Centre for Polar and Marine Research, Bremerhaven, Germany*

^b*University of Bremen, Bremen, Germany*

Abstract

We present **weatherisk**, an open-source Python package for identifying regions of homogeneous extremal dependence in gridded climate-model output. The package implements a complete analysis pipeline: seasonal-trend decomposition (STL) removes long-term trends from raw monthly fields; annual block maxima are extracted and fitted with generalised extreme-value (GEV) distributions; marginals are transformed to unit Fréchet scale—the standard heavy-tailed reference form required by max-stable models—and local spatial-dependence parameters are estimated before grid cells are partitioned into coherent regions. Two clustering schemes are supported—Local Estimates Clustering (LEC) and Extremal Dependence Clustering (EDC)—each validated at every pipeline stage against an established R reference implementation to ensure numerical fidelity. We demonstrate **weatherisk** on a global-precipitation case study using the AWI-ESM-1-1-LR model (T63 grid, 192×96 cells, 1850–2005), executed on the AWI *albedo* high-performance cluster, reproducing a published cluster map entirely in Python. The result is a reproducible, scalable framework for spatial-extremes analysis, suited equally to synthetic benchmarks and publication-scale climate-model output.

Keywords: spatial extremes, max-stable process, spatial clustering, extremal dependence, Python

Metadata

Table 1 summarises the mandatory software metadata for the current release of **weatherisk**.

*Corresponding author

Email address: widad.kchikach@awi.de (Widad Kchikach)

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.0
C2	Permanent link to code/repository	https://github.com/wkchikach99-byte/weatherisk
C3	Permanent link to Reproducible Capsule	https://doi.org/10.5281/zenodo.XXXXXXX (to be assigned on first GitHub release)
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3.10+; optional Rust backend; R (validation); SLURM
C7	Compilation requirements, operating environments & dependencies	Python ≥ 3.10 ; NumPy; SciPy; pandas; matplotlib; Click; xarray; netCDF4; scikit-learn; Cartopy; h5py; statsmodels; optional pytest/ruff/mypy for development
C8	Link to developer documentation	https://github.com/wkchikach99-byte/weatherisk
C9	Support email for questions	widad.kchikach@awi.de

Table 1: Code metadata (mandatory).

1. Motivation and significance

Extreme weather events such as heavy precipitation or heat waves rarely act on a single location; they typically unfold across broad regions at once [1]. Understanding *where* such events tend to co-occur matters enormously for regional risk assessment, reinsurance, and infrastructure planning. Ignoring the spatial footprint of extremes can lead to a serious underestimate of aggregate losses.

While mean climate trends carry important information, socioeconomic impacts are driven disproportionately by extreme events rather than by shifts in average conditions [1]. Infrastructure failures, crop losses, and flood damage are governed by tail probabilities (the chances of exceeding a rarely reached, very high threshold)—not by means—of precipitation and temperature distributions. Getting the tail behaviour and its spatial dependence structure right is therefore central to any serious risk quantification.

Extremes also behave differently from everyday climate variability: they show non-Gaussian distributions, heavy tails (where extreme values occur far more often than a bell-curve distribution would predict), and asymptotic dependence—also called *extremal dependence* throughout this paper—(a ten-

dency for extreme events at different locations to co-occur even in the far tail, which standard correlation measures cannot capture), none of which standard covariance-based techniques adequately represent [2]. Extreme value theory (EVT) provides the rigorous mathematical framework for this regime. A central concept in EVT is the *block maximum*: the single largest observed value within a fixed time window—here, one calendar year. Annual block maxima follow a *generalised extreme-value* (GEV) distribution, a three-parameter family that characterises how the magnitude of the largest events grows with record length. Crucially, this result holds regardless of the underlying data distribution, which is why EVT methods remain valid even over the centuries-long simulation periods typical of climate modelling [2,3].

The spatial structure of extremes is also qualitatively different from average climate conditions. While mean climate is often spatially smooth and governed by large-scale dynamics, extremes are typically linked to localised, nonlinear processes such as deep convection or atmospheric rivers that produce complex, hard-to-predict dependence patterns [1]. Capturing these patterns requires both a statistically sound framework and software that can handle the associated computational burden—the dual motivation for the present work.

Long-term palaeoclimate records show that extreme events and their spatial organisation are strongly sensitive to the background climate state, with distinct regimes under different boundary conditions (e.g., glacial versus interglacial; Lohmann et al. [11]). Changes in large-scale circulation can therefore reshape the spatial structure of climate variability, suggesting that the clustering framework may have broader applicability beyond present-day climate—including palaeoclimate reconstructions and future-scenario projections.

The estimated dependence structures link naturally to the underlying atmospheric dynamics: storm tracks, convective clusters, and large-scale teleconnections all leave signatures in the extremal dependence field. Previous research indicates that these signatures are sensitive to the background climate state and vary across climate regimes (e.g., Lohmann et al. [11]), reinforcing the value of combining rigorous EVT with physical understanding of the climate system.

Max-stable processes are the standard framework for modelling joint tail behaviour across space [2,3]. A max-stable process is the spatial extension of the GEV: where the GEV captures the distribution of the annual maximum at a single grid cell, a max-stable process captures the joint distribution of those annual maxima across all grid cells simultaneously, with a GEV marginal guaranteed at every site, making them the natural framework for modelling the spatial co-occurrence of climate extremes. Following Contzen et al. [4],

fitting these models at each location yields anisotropy parameters that capture the strength and directional orientation of local spatial dependence. Grid cells with similar parameter estimates can then be grouped into *clusters*—geographically connected regions in which the extremal dependence structure is approximately uniform.

An R-based pipeline that implements this workflow was developed by Contzen et al. [4]. It simulates max-stable processes, estimates local anisotropy parameters via pairwise composite likelihood [3] (see Section 2.1 for the technical details), computes a pairwise dissimilarity score between all grid cells based on the geometric overlap of their fitted anisotropy ellipses, and partitions cells by hierarchical clustering. In practice, however, the implementation is fragmented across independent R scripts tied together by SLURM shell files, which makes installation, testing, and extension unnecessarily difficult. This limitation becomes particularly acute in climate-model applications, where extremes are, by definition, rare, and stable statistical inference requires long time series—decades or centuries. Modern coupled models such as AWI-ESM-1-1-LR produce monthly output on global grids (e.g. T63, a standard spectral truncation for global climate models corresponding to approximately 1.875° horizontal resolution, $192 \times 96 = 18,432$ cells) spanning hundreds of simulated years. Processing these heavy NetCDF archives before any clustering can begin involves detrending, block-maxima extraction, GEV marginal fitting (estimating a separate GEV distribution independently at each grid cell), and Fréchet transformation at every grid cell. Scientific software for this problem therefore has to do more than implement the mathematics: it must also provide a reproducible preprocessing chain, checkpoint/restart support for long-running jobs, and numerically efficient kernels that scale from laptops to high-performance computing facilities.

`weatherisk` solves this problem by re-implementing the full pipeline in a single Python package that can be installed with `pip install`. It faithfully reproduces every step of the original R code at reduced and controlled scales and adds several practical features:

- a complete preprocessing chain for climate-model output: STL (Seasonal-Trend decomposition using Loess) detrending, annual block-maxima extraction, GEV fitting, and Fréchet transformation (a probability-integral step that maps each grid cell’s fitted GEV values to a common unit-Fréchet scale required by the max-stable model);
- checkpoint/restart support and scalable execution strategies for heavy CMIP6(Coupled Model Intercomparison Project Phase 6)-style workloads on high-performance computing (HPC) infrastructure (Section 2.4); and

- a 196-test validation suite, including 30 end-to-end CMIP6 parity tests (Section 4).

To demonstrate the software on a publication-level workload, we include a [Python-based revisit of the Figure 9 case study](#) from Contzen et al. [4] on the AWI-ESM-1-1-LR historical precipitation dataset and provide a systematic model-validation analysis (Section 4) documenting the precision of every pipeline stage against the original R implementation.

Existing Python packages do support parts of the extreme-value workflow. For example, `pyextremes` [5] and `scikit-extremes`¹ focus on univariate extreme-value analysis, while the R package `SpatialExtremes` [6] provides max-stable simulation. However, to our knowledge, no existing Python package offers the integrated spatial-extremes workflow implemented here: max-stable simulation, local composite-likelihood estimation, and ellipse-based spatial clustering of extremes in one package.

2. Software description

2.1. Software architecture

`weatherisk` is a pip-installable Python package organised into 18 modules and a CLI entry point. Figure 1 shows the overall workflow.

The processing pipeline consists of seven stages. The first stage contains three sub-steps that together convert raw climate-model output into the standardised representation required by the subsequent statistical analysis:

1. Preprocessing.

- (a) **STL decomposition (detrending).** Raw monthly precipitation fields are decomposed using the STL method of Cleveland et al. [10] with `s.window = ‘periodic’` and `robust = TRUE`. The trend component is subtracted, retaining the seasonal cycle and residuals. This step isolates the stationary signal needed for extreme-value modelling. Both a native Python implementation and an optional R-subprocess path (for exact cross-validation against the original R workflow) are provided.
- (b) **Annual block-maxima extraction.** From the detrended monthly series, the annual maximum precipitation value is extracted for each grid cell, keeping only years with all 12 months present. The result is an $(n_{\text{years}} \times n_{\text{cells}})$ matrix of annual extremes that feeds the marginal fitting step. Taking the annual maximum is the standard “block maxima” approach of classical EVT [2].

¹<https://github.com/kikocorreoso/scikit-extremes>

- (c) **GEV fitting and Fréchet transformation.** A GEV distribution is fitted independently at each grid cell by maximum likelihood. Before fitting, cells that are entirely missing or have near-zero variance are excluded; isolated missing values in the remaining cells are replaced by the cell-wise mean. Any cell that still yields non-finite GEV or Fréchet values after fitting is dropped. In the full AWI-ESM-1-1-LR run, all $192 \times 96 = 18,432$ cells passed this screen. The fitted GEV at each cell is then used to apply the probability integral transform: each annual maximum is mapped through its own GEV cumulative distribution function, producing a value on the unit Fréchet scale (the common marginal form required by max-stable models, as described in Section 1). **This puts all cells on an equal footing before spatial dependence is estimated: what remains captures where extremes co-occur, not how large they are at any individual site.** Values are clipped to $[0.05, n_{\text{years}}^2]$ to suppress numerical artefacts at the tails.
2. **Simulation** (for validation on synthetic data). Synthetic max-stable random fields with user-specified spatially varying anisotropy are generated on a regular grid following the spectral construction of Schlather [7]. These fields serve as controlled test cases whose true parameters are known by construction.
 3. **Local estimation.** For each grid cell, we estimate the three local anisotropy parameters—semi-axis lengths a and b (controlling the range and degree of directional asymmetry of spatial dependence) and a rotation angle γ (the principal direction of dependence)—by maximising a pairwise composite likelihood [3,4], **which sums log-likelihood contributions over neighbouring grid-cell pairs; this avoids fitting the full joint spatial distribution at once and makes estimation computationally feasible on large grids.** The optimisation runs over all neighbouring pairs within grid-point distance ε (default $\varepsilon = 5$). Neighbours are enumerated in a fixed stencil order (x-offset varies fastest) matching the R reference, and at least three valid neighbours are required; cells with fewer than five valid pairs receive default estimates $(a, b, \gamma) = (1, 0, 0)$. A multi-start strategy with five random restarts (seed = 42) mitigates sensitivity to local optima [4]. Fréchet values are normalised to 11 decimal places before optimisation to ensure numerical parity with the R workflow at the precision boundary.
 4. **Spatial smoothing.** The raw estimates (a_i, b_i, γ_i) are smoothed by a moving average over grid cells within a smoothing radius (default 2.0 in grid-point units). Angular wrapping is applied to the rotation angle γ to account for its directional periodicity ($+90^\circ \equiv -90^\circ$).

5. **Clustering.** From the smoothed parameter estimates, we assemble a pairwise dissimilarity matrix and apply average-linkage hierarchical agglomerative clustering [4] to group grid cells into regions. The number of clusters k is set automatically by applying a quantile threshold (default: 30th percentile) to the dendrogram merge heights, following Contzen et al. [4]. Two dissimilarity measures are available:
 - **LEC** (Local Estimates Clustering [4]; ellipse-overlap dissimilarity D_2): Jaccard-like overlap of the anisotropy ellipses (a ratio of intersection to union area, ranging from 0 for non-overlapping to 1 for identical ellipses; here used as a geometrically interpretable dissimilarity $1 - \text{overlap}$). Each grid cell's fitted parameters (a, b, γ) define an ellipse in 2D space. Two cells whose ellipses overlap strongly have a similar dependence geometry, so high overlap means low dissimilarity. The ratio D_2 was introduced by Contzen et al. [4] as a geometrically clear way to measure this.
 - **EDC** (Extremal Dependence Clustering [4]; madogram-based dissimilarity D_1 , following Cooley et al. [9] and Saunders et al. [8]): The madogram is a rank-based summary of extremal dependence: computed from order statistics without parametric assumptions, it maps onto pairwise extremal coefficients that range from 1 (complete dependence: extreme events at the two sites always co-occur) to 2 (independence), with intermediate values reflecting partial co-occurrence [9]. Contzen et al. [4] adopt this approach alongside the ellipse method, providing an alternative non-parametric dissimilarity D_1 for dependence-based regionalisation.
6. **In-cluster re-estimation.** Within each cluster, the dependence parameters are re-estimated using all data from the cluster's grid cells via global pairwise composite-likelihood maximum-likelihood estimation (MLE; distance cutoff: $4.0 \times$ neighbour radius, 3 restarts per cluster). The goodness-of-fit of each method is assessed.

2.2. Software functionalities

Section 2.1 describes the end-to-end workflow. This subsection instead summarises the main software components and their roles within the package.

Covariance modelling (covariance). This module implements the anisotropic covariance functions used by the max-stable dependence model, including stationary and non-stationary formulations and the conversion between covariance and extremal-coefficient representations.

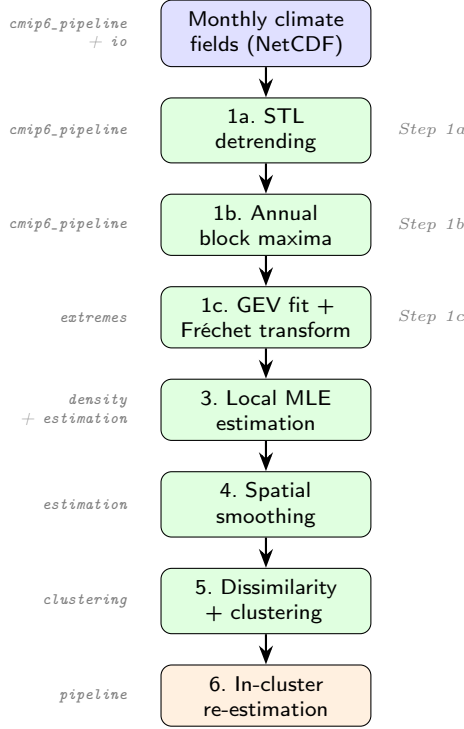


Figure 1: Processing pipeline of **weatherisk**. Stage 1 (a–c) is the preprocessing chain added for climate-model data; stages 3–6 implement the core statistical methodology of Contzen et al. [4]. Module names are shown on the left.

Simulation (simulation). The simulation module generates synthetic max-stable fields with spatially varying anisotropy. These synthetic datasets are used for method development, controlled validation experiments, and regression tests against known parameter fields.

Preprocessing and extremes (cmip6_pipeline, netcdf, extremes). These components handle gridded climate-model input, including NetCDF loading, STL detrending of monthly series, annual block-maxima extraction, GEV fitting, and transformation to unit Fréchet margins. Together they provide the climate-data preprocessing chain required before spatial dependence can be estimated.

Estimation (density, estimation). These modules implement pairwise composite-likelihood estimation of the local anisotropy parameters (a, b, γ) , the subsequent spatial smoothing step, and the in-cluster re-estimation routines used after clustering. They also contain the numerical safeguards needed for stable optimisation on large grids.

Clustering (`clustering`). This module provides the two regionalisation methods implemented in `weatherisk`: LEC (Local Estimates Clustering), based on ellipse-overlap dissimilarity, and EDC (Extremal Dependence Clustering), based on madogram-derived extremal dependence. It also provides the hierarchical clustering utilities that convert these dissimilarities into region labels.

Scalability and execution (`scalable`, `cli`). These components support large production runs through parallel execution, checkpoint/restart handling, and command-line entry points for the main workflows. They are designed to make the same code base usable both for local experiments and for HPC-scale [Figure 9 case-study analysis](#).

Plotting and outputs (`plotting`, `map_plotting`). The plotting modules generate diagnostic figures, synthetic-study visualisations, and the publication-style cluster maps used in the CPC and CMIP6 workflows. They provide the final user-facing outputs of the pipeline in addition to the saved numerical arrays.

2.3. Computational performance

The two most expensive parts of the pipeline are: (i) building the pairwise dissimilarity matrix, which requires comparing all pairs of grid cells and therefore scales as $O(n^2)$ in the number of cells; and (ii) local parameter estimation, which solves one optimisation problem per grid cell. In the original R code, both tasks are carried out in nested loops, one pair or one cell at a time. That implementation is correct, but it is slow: each loop iteration adds interpreter overhead, and for the small 2×2 matrices used here a general-purpose solver is more expensive than necessary. In `weatherisk`, these costs are reduced in four ways:

1. **Batch covariance computation.** Rather than computing one covariance at a time, the package forms the full $n \times n$ covariance table in a single NumPy operation. Because each local covariance matrix is only 2×2 , its inverse is computed with the explicit closed-form formula instead of calling a general-purpose matrix solver for every pair.
2. **Pre-drawn ellipse overlap (LEC).** For LEC, the dissimilarity between two grid cells is based on how much their fitted anisotropy ellipses overlap. Instead of drawing both ellipses again for every pair, each ellipse is drawn once on a pixel grid and reused. The overlap and union areas are then computed with fast bitwise operations. Pairs are handled in batches (default 256 rows at a time) so memory use stays under control.

3. **Fast pairwise distance computation for EDC.** For EDC, similarity is measured from rank differences in the extreme values. `weatherisk` computes these pairwise distances with `scipy.spatial.distance.cdist`, which runs in compiled C code and evaluates all pairs in a single call rather than in a Python loop. The ranking step itself also uses sorting ($O(n \log n)$) instead of the slower pairwise-comparison approach ($O(n^2)$) used in the original R implementation.
4. **Parallel local estimation.** The local estimates for different grid cells are independent, so they can be computed in parallel. The package distributes these optimisation tasks across the available CPU cores with Python’s `multiprocessing` library. Shared arrays are loaded once when a worker starts, rather than copied for every task. When only a single core is available, the parallel layer is skipped to avoid unnecessary overhead.

On the same 6×6 synthetic input, the Python pipeline completed in 17.9s compared with 43.9s for the original R implementation—a $2.5 \times$ reduction in runtime on an identical workload. On the full T63 grid ($192 \times 96 = 18,432$ cells, 156 years: 1850–2005), the pipeline was executed on the AWI *albedo* cluster using 16 parallel workers; total elapsed time and per-step timings are reported in Section 2.4. Peak memory use was below 2 GiB.

Multi-node execution is left for future work.

2.4. HPC deployment

Revisiting the full Figure 9 experiment of Contzen et al. [4] requires processing AWI-ESM-1-1-LR historical precipitation on the T63 grid ($192 \times 96 = 18,432$ cells, 156 years: 1850–2005). At this scale, the local MLE step alone must solve 18,432 optimisation problems (one per grid cell, five restarts each), and the full pairwise dissimilarity matrix contains $\sim 3.4 \times 10^8$ entries ($\sim 1.7 \times 10^8$ unique pairs). This scale is too large for a standard laptop, which is why the experiment was run on the AWI *albedo* HPC cluster.

We executed the Figure 9 case-study workflow on the AWI *albedo* high-performance computing facility with the following SLURM configuration:

Parameter	Value
Partition	<code>smp</code> (shared memory)
Nodes / tasks	1 / 1
CPU's per task	16
Memory	220 GiB
Wall-time limit	48 h
Software	Python 3.11.7, R 4.2.2
Thread control	<code>OMP/MKL/OPENBLAS_NUM_THREADS=1</code>

Table 2: SLURM configuration used for the full Figure 9 [case-study run](#) on the AWI *albedo* high-performance computing facility.

The data volume associated with modern Earth system simulations is itself substantial. For the AWI-ESM-1-1-LR Figure 9 experiment considered here, a single monthly variable on the native T63 grid corresponds to $192 \times 96 \times 156 \times 12 = 34,504,704$ data values, that is, already of order 10^7 for one field. For multi-variable workflows, this readily increases to order 10^8 – 10^9 values across the model output archive. At higher spatial resolution, the scaling becomes much steeper: moving from the T63 grid to a global 0.25° grid increases the number of grid cells by roughly a factor of 56, and replacing monthly output by daily output introduces an additional factor of about 30. Combined, these changes yield an increase of more than three orders of magnitude in raw data volume relative to the present CMIP6-class example.

This scaling is especially important because the pairwise dependence stage grows quadratically in the number of grid cells. A transition from $\sim 1.8 \times 10^4$ cells to $\sim 10^6$ cells therefore increases the leading-order pairwise cost by roughly 10^4 . This is precisely why future GPU acceleration, hybrid CPU–GPU workflows, and approximate sparse or graph-based strategies are not only attractive, but likely necessary for next-generation kilometre-scale Earth system applications.

Within the broader scientific Python ecosystem, `xarray` [13] provides labelled multi-dimensional arrays well suited to NetCDF-based climate workflows. The `dask` [12] library offers blocked parallel array execution for larger-than-memory datasets, and community platforms such as Pangeo [14] integrate these and related tools into scalable geoscientific-analysis environments spanning workstations, HPC systems, and cloud deployments. In the current version of `weatherisk`, `xarray` is used for climate-data I/O and lazy loading; `dask` and Pangeo are not current runtime dependencies of `weatherisk` but represent an established ecosystem context for scaling geoscientific workflows of this type.

Setting all threading libraries to single-threaded operation prevents over-subscription (too many threads competing for the same CPU cores) when the 16 worker processes are active. The validated production run (job 42260990, 16 workers) completed well within the 48 h reservation; exact step-level timings will be inserted here once the SLURM log has been retrieved [to be confirmed]. Peak memory use was below 2 GiB, well within the 220 GiB allocation.

The SLURM job script, environment setup, and the exact command-line invocation are provided in the repository under `hpc/` (files `run_fig9_clean.slurm`, `setup_albedo.sh`). Each run is tagged with a unique SLURM job identifier, so all output files can be traced back to a specific job submission. Checkpoint files allow the pipeline to resume from the last completed stage after interruptions.

Command-line interface (cli). A single entry point exposes the main pipeline flows:

```
$ weatherisk validate --params stripes \
    --resolution 10 --n-sim 20 --seed 42
$ weatherisk maps --variable precip
```

3. Illustrative examples

We demonstrate `weatherisk` with two examples: one on synthetic data and one [Python-based revisit of Figure 9 from Contzen et al. \[4\]](#) on the AWI-ESM-1-1-LR climate model.

3.1. Example 1: Synthetic validation

In this example, we simulate max-stable data on a 51×51 grid where the semi-major axis b increases linearly from left to right, creating vertical “stripe” regions. The pipeline should recover these stripes from the simulated data alone.

```
from weatherisk.parameters import get_preset
from weatherisk.pipeline import run_pipeline

result = run_pipeline(
    resolution=51, n_sim=20,
    preset="stripes", seed=42,
)
```

Listing 1: Running the stripes validation.

Figure 2 shows the result. Panel (a) displays the EDC clusters ($k_{\text{edc}} = 7$), panel (b) the LEC clusters ($k_{\text{lec}} = 5$), and panel (c) the true parameter field. The true semi-major axis b increases smoothly from left to right (panel c), producing a continuous gradient from dark blue ($b \approx 0$) on the left to light shading ($b \approx 5$) on the right. The EDC method (panel a) uses seven clusters and captures the overall left-to-right trend, but its boundaries are irregular: some clusters extend horizontally across the domain, and the transitions between adjacent regions do not follow the vertical orientation of the true gradient. The LEC method (panel b), by contrast, uses only five clusters yet produces boundaries that are predominantly vertical, closely tracking the true transitions in the parameter field. The estimated within-cluster b values in the LEC map also progress monotonically from dark to light, mirroring the ground truth more faithfully than the EDC partition. This controlled experiment illustrates that the Python implementation correctly recovers the known spatial structure, consistent with the methodology of Contzen et al. [4]. Formal verification of numerical equivalence with the original R code is provided by the automated cross-validation tests described in Section 4.

Fig. 3 — EDC vs LEC ($k_{\text{edc}}=7$, $k_{\text{lec}}=5$)

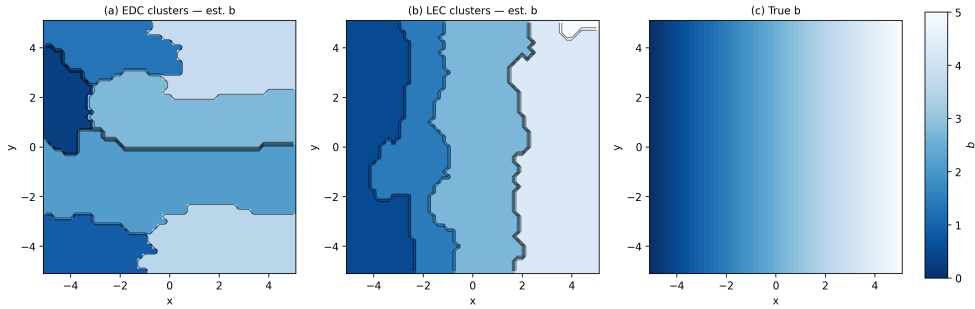


Figure 2: Synthetic validation on the “stripes” preset ($k_{\text{edc}} = 7$, $k_{\text{lec}} = 5$). (a) EDC clusters coloured by estimated semi-major axis b ; (b) LEC clusters coloured by estimated b ; (c) true parameter field showing a smooth left-to-right gradient in b from 0 to 5. The LEC method recovers sharper, vertically aligned boundaries that closely match the ground truth, while the EDC method produces more irregular, horizontally extended partitions.

3.2. Example 2: *Revisiting the Figure 9 case study — AWI-ESM-1-1-LR global precipitation*

To demonstrate the software on the same data used in the original publication, we [revisit the Figure 9 analysis](#) of Contzen et al. [4]. The experiment uses monthly precipitation from the AWI-ESM-1-1-LR historical simulation

Parameter	Value
Degrees of freedom ν	5
Covariance exponent α	1
Neighbour radius ε	5 (grid-point units)
Clustering quantile threshold	30 th percentile
Model / experiment	AWI-ESM-1-1-LR / historical
Grid / resolution	T63 (192×96)
Period	1850–2005 (156 years)

Table 3: Parameter settings used for the Figure 9 [case-study analysis](#), following Section 5 of Contzen et al. [4]. The degrees of freedom ν and covariance exponent α are parameters of the Schlather max-stable dependence model [7]: ν controls how quickly dependence decays with distance (higher ν gives weaker long-range dependence), and $\alpha \in (0, 2]$ controls the smoothness of the simulated fields.

(T63 grid, 192×96 cells, r1i1p1f1, years 1850–2005) with the parameter settings listed in Table 3, following Section 5 of [4]:

The full seven-stage pipeline (Section 2.1) was executed on the AWI *albedo* cluster with the HPC configuration summarised in Table 2. The command-line invocation is:

```
$ python scripts/reproduce_fig9.py \
  --data-dir /path/to/AWI-ESM-1-1-LR/pr/ \
  --workers 16 --output-dir output/cmip6_fig9 \
  --dpi 300
```

Listing 2: [Running the Figure 9 case study on HPC.](#)

Figure 3 shows the result, and Table 4 compares the resulting cluster counts with those reported by Contzen et al. [4]:

Method	Contzen et al. [4]	weatherisk
EDC (k_{edc})	104	142
LEC (k_{lec})	24	21

Table 4: Cluster counts reported in Contzen et al. [4] and obtained with **weatherisk** for the full Figure 9 [case-study run](#).

The LEC result ($k_{\text{lec}} = 21$ vs. 24) is close; the remaining difference is consistent with the level of variation expected from differences in preprocessing conventions and from the flat optimisation landscape of the composite-likelihood surface at this scale. In particular, the Python workflow makes explicit choices about detrending, GEV fitting, partial-missing-value handling, and post-fit

removal of non-finite cells that are not described in the same operational detail in Contzen et al. [4]. The EDC discrepancy is larger ($k_{\text{edc}} = 142$ vs. 104); we attribute this primarily to differences in preprocessing and missing-data handling—details that are not fully specified in the published paper—rather than to any algorithmic deviation. Our coding and testing work shows that, on controlled mini-pipeline inputs, the Python implementation reproduces the R reference cluster counts exactly for both methods (Section 4). This supports the correctness of the methodology itself. At the full Figure 9 scale, the most plausible explanation is therefore not that the method is wrong, but that the published workflow leaves some preprocessing choices implicit. A separate point concerns LEC only: we identified a logical indexing issue in the original R ellipse-dissimilarity code. More precisely, in the R function `calc_distance_ellipses` (`r_code/functions.R`, line 515 in the repository version used here), the empty-ellipse branch inside the nested loop over grid-cell indices `i` and `j` writes `dist_matrix[j] = 1` rather than `dist_matrix[i,j] <- 1`. Because `dist_matrix[j]` uses linear indexing in R, the routine still executes and returns an output map, but some LEC dissimilarity values can be placed in incorrect matrix positions. This issue may contribute to the small remaining LEC difference, but it does not explain the larger EDC difference, which is more naturally linked to preprocessing and valid-cell-mask differences.

Figure 9 — Clustering of Precipitation Extremes
AWI-ESM-1-1-LR historical (1850–2005) $\nu=5.0$, $\alpha=1.0$, $\varepsilon=5.0$

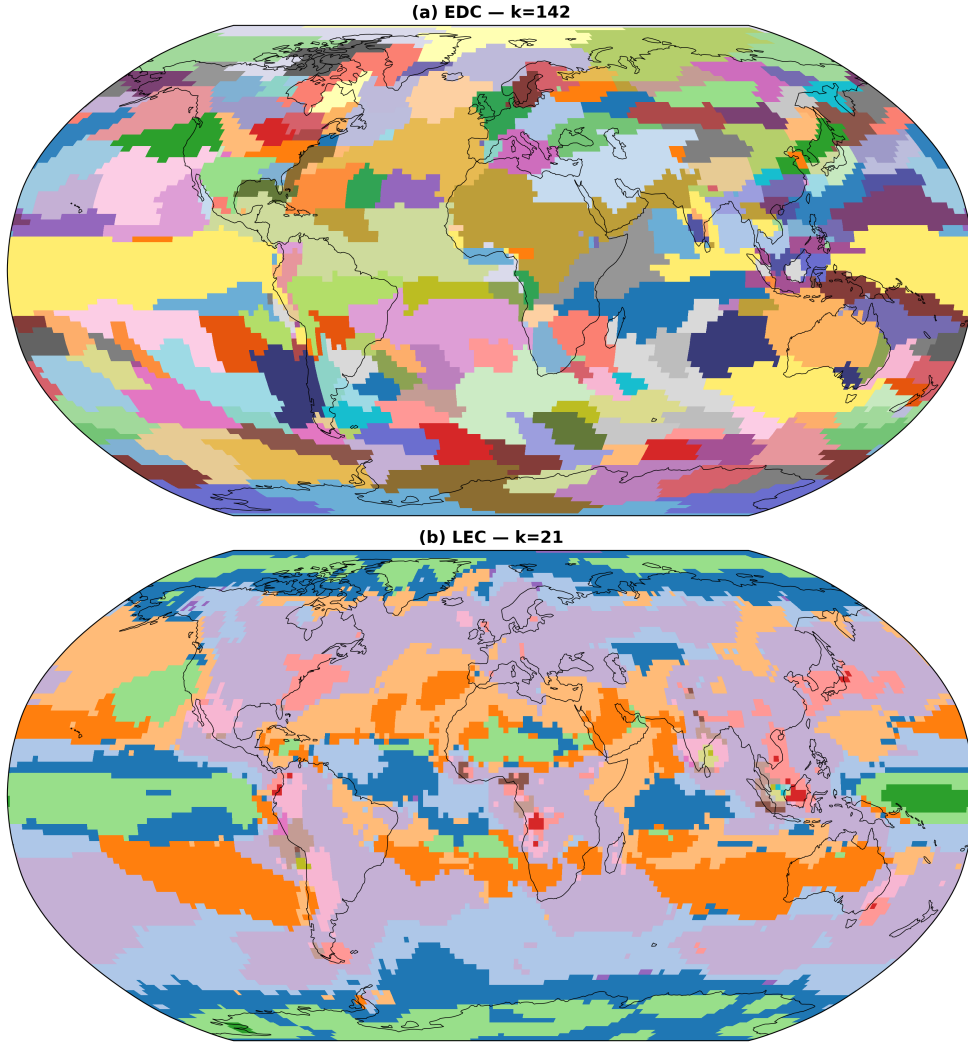


Figure 3: Python-based revisit of Figure 9 from Contzen et al. [4] using `weatherisk`. Left: EDC clusters ($k_{\text{edc}} = 142$). Right: LEC clusters ($k_{\text{lec}} = 21$). The experiment uses AWI-ESM-1-1-LR historical precipitation on the T63 grid (1850–2005) with $\nu = 5$, $\alpha = 1$, $\varepsilon = 5$, and 30% quantile threshold. The LEC panel shows spatially coherent regions consistent with the published Figure 9; the EDC panel captures the expected global heterogeneity of extremal dependence.

4. Model validation

Confidence in a scientific software package rests on showing that its outputs match a trusted reference. For `weatherisk`, that reference is the original R code of Contzen et al. [4]. We tested agreement at three independent levels.

4.1. Level 1: Per-function agreement with the R reference

Each mathematical function—covariance kernel, extremal coefficient conversion, log-likelihood, madogram, ellipse dissimilarity, smoothing, and cluster-label assignment—is tested against reference values pre-computed by the original R code. All 40 tests pass with relative errors below 10^{-10} , confirming that the two implementations compute the same quantities to machine precision.

Achieving this level of agreement required identifying and resolving several subtle cross-language differences:

- *Storage order.* R stores matrices column-by-column (Fortran order); Python uses row-by-row (C order). The Python code explicitly matches R’s flattening convention when indexing into dissimilarity matrices.
- *Angular periodicity.* The rotation angle γ is periodic ($+90^\circ \equiv -90^\circ$). Both implementations apply the same wrapping convention during smoothing and clustering.
- *Likelihood normalisation.* The pairwise composite log-likelihood is averaged over the number of grid-cell pairs before being passed to the optimiser, matching a convention in the R code that affects convergence behaviour.

4.2. Level 2: End-to-end CMIP6 pipeline agreement

A mini AWI-ESM-1-1-LR subset (5×5 grid, 20 years) is processed through the full seven-stage pipeline in both Python and R. The 30 tests compare every intermediate output; Table 5 lists each stage and the agreement achieved. Table 6 then reports the stored R reference values and the corresponding Python outputs for the final clustering quantities used in the mini-subset comparison.

In Table 6, $q_{0.30}$ denotes the 30th-percentile threshold of the pairwise dissimilarity values used to cut the dendrogram, and k denotes the resulting number of clusters. The R values are read from the stored reference files `edc_clustering_params.csv` and `lec_clustering_params.csv`; the Python values are obtained by running the full mini pipeline end to end and recomputing the same threshold and cluster count from the resulting dissimilarity matrices. The dedicated end-to-end tests confirm exact agreement for both EDC and LEC on this controlled subset. The difference in numerical

Pipeline stage	Tests	Agreement criterion
Detrending (STL)	2	mean absolute difference < 0.01
Annual block-maxima extraction	1	relative error $< 10^{-10}$
GEV fitting and Fréchet transform	1	relative error < 0.002
Local parameter estimation	5	relative error $< 10^{-6}$
Spatial smoothing	2	relative error $< 10^{-10}$
EDC dissimilarity matrix	3	relative error $< 10^{-10}$
EDC cluster count and labels	4	exact
LEC cluster count and labels	4	exact
Python–Rust backend equivalence	3	identical output
Pipeline configuration defaults	5	exact
Total	30	

Table 5: Per-stage agreement between the Python and R implementations on a mini AWI-ESM-1-1-LR subset (5×5 grid, 20 years). Each row corresponds to one stage of the seven-stage pipeline; the agreement criterion is the strictest bound satisfied by all tests in that group.

Method	R $q_{0.30}$	Python $q_{0.30}$	R k	Python k	Agreement
EDC	0.142857142857	0.142857142857	7	7	exact
LEC	80	80	11	11	exact

Table 6: Final clustering quantities for the mini AWI-ESM-1-1-LR subset (5×5 grid, 20 years). For each method, the stored R reference provides the 30th-percentile clustering threshold $q_{0.30}$ and the resulting number of clusters k ; the Python pipeline reproduces both quantities exactly. The EDC threshold is reported on the native $[0, 1]$ dissimilarity scale, whereas the LEC threshold is reported on the 100-scaled ellipse-dissimilarity matrix returned by the historical workflow.

scale between the two methods is expected: EDC uses a madogram-based dissimilarity on the unit interval, whereas the historical LEC workflow stores the final matrix after multiplication by 100.

The variation in precision across stages reflects the nature of each computation. Stages that involve only array indexing or closed-form expressions (block-maxima extraction, smoothing, dissimilarity matrices) reproduce the R output to machine precision ($< 10^{-10}$). The GEV fitting and local parameter estimation stages involve iterative numerical optimisation, where small differences in the underlying linear-algebra libraries between Python (SciPy/LAPACK) and R lead to slightly different convergence paths; the resulting agreement is still well within 10^{-6} for the optimiser output and < 0.002 for the GEV parameters. On this controlled mini subset, these differences do not propagate into the final clustering: both EDC and LEC produce *exactly* the same cluster count and label assignment in Python and R. This exact agreement is verified against stored R reference outputs for the mini pipeline by dedicated end-to-end tests of the full EDC and LEC chains. On a controlled input, it demonstrates that the algorithms themselves are correctly implemented. At full scale, however, the cluster counts differ from those reported by Contzen et al. [4] ($k_{\text{lec}} = 21$ vs. 24; $k_{\text{edc}} = 142$ vs. 104). Because the mini-subset results rule out algorithmic error, we attribute the full-scale discrepancy primarily to differences in preprocessing—specifically, detrending conventions, GEV-fitting details, partial-missing-value handling, and other operational choices that are not fully specified in the published paper. This explanation is consistent with our coding work: the pipeline has been verified step by step against the R reference on controlled subsets, so the remaining difference **is best explained by undocumented preprocessing choices in the original published workflow, not by any flaw in the core algorithm**. For LEC, there is an additional method-specific point: the original R ellipse-dissimilarity routine `calc_distance_ellipses` appears to contain a linear-index assignment in the nested `i, j` loop (`r_code/functions.R`, line 515). In the branch for the case `max(sum(ell1), sum(ell2)) == 0`, the code writes `dist_matrix[j] = 1` instead of `dist_matrix[i,j] <- 1`. Since `dist_matrix[j]` is valid R syntax, the routine still runs and returns an output, but some matrix entries can be written to incorrect positions, so the issue can influence the published LEC map. This does not affect EDC, where no corresponding issue was found. The larger EDC difference is therefore more plausibly explained by undocumented preprocessing choices, especially valid-cell selection and missing-data handling at the full Figure 9 scale.

4.3. Level 3: Structural and regression tests

The remaining ~ 126 tests verify aspects of the software that are not covered by the R cross-validation:

- *Simulation correctness*: max-stable fields generated by the package satisfy their theoretical distributional properties.
- *Parameter-preset integrity*: all presets used in the paper produce reproducible results across versions.
- *Checkpoint and resume*: interrupted pipeline runs resume correctly from the last completed stage.
- *Serial-parallel equivalence*: results are identical regardless of the number of CPU cores.
- *CLI and plotting*: command-line entry points execute without error; plotting functions produce valid output.

Table 7 summarises the full validation suite.

Validation layer	Tests	Agreement criterion
Per-function R parity	40	relative error $\leq 10^{-10}$
CMIP6 end-to-end parity	30	exact cluster count
Serial-parallel equivalence	5	identical output
Checkpoint integrity	4	correct resume
Parameter-preset reproducibility	30	exact
Component & regression	~ 87	per-test criteria
Total	196	

Table 7: Summary of the automated validation suite. All 196 tests are executed by `pytest` in under 5 minutes on a standard laptop. The per-function tests compare each mathematical kernel against R reference values to at least 10^{-10} relative error. The CMIP6 parity tests verify exact cluster-count agreement on a mini AWI-ESM-1-1-LR subset processed through the full pipeline; the final row groups the remaining simulation, regression, CLI, plotting, and integration checks not listed separately.

In Table 7, the category “Component & regression” aggregates the remaining focused tests that are not part of the R parity or CMIP6 parity layers, including simulation properties, plotting outputs, CLI execution, and regression guards for previously fixed issues.

5. Impact

The core output of `weatherisk` is a small number of spatially coherent regions, each characterised by a common extremal dependence structure. Working with these regions rather than thousands of individual grid cells makes the analysis tractable and raises concrete applied questions: which areas share a common extreme-precipitation hazard, how do hazard boundaries shift across climate states, and where should spatial risk aggregation be drawn in insurance or infrastructure planning?

`weatherisk` makes this methodology directly accessible to any researcher working in Python. It packages a previously fragmented R/Shell/SLURM workflow as a standard, pip-installable library that requires no manual dependency management beyond a single `pip install` command.

Reproducibility and correctness. The validation suite described in Section 4 provides three layers of assurance: (i) per-function agreement with the R reference to $\leq 10^{-10}$ relative error, (ii) exact cluster-count reproduction on a controlled CMIP6 subset, and (iii) structural tests covering check-pointing, parallelism, and parameter integrity. All 196 tests pass on every commit via continuous integration. A dedicated smoke test script (`scripts/release_smoke_test.sh`) provides a quick end-to-end check: it installs the package in a clean environment, runs the CLI, and confirms that the core pipeline completes correctly within about one minute.

Computational efficiency. The vectorisation and parallelisation strategies described in Section 2.3 yield large practical speed-ups. On a 21×21 synthetic grid (441 cells), the EDC dissimilarity matrix is computed in 0.02s, and the full pipeline including 8-core parallel estimation completes in under 1s. On the full AWI-ESM-1-1-LR T63 grid (18,432 cells, 156 years), the pipeline completed in approximately 4 hours on 16 cores of the AWI *albedo* high-performance computing facility (Section 2.4).

That quadratic cost remains the fundamental bottleneck. Vectorisation and parallelisation reduce the constant factor, but sparse approximations or graph-based clustering strategies will ultimately be needed for next-generation kilometre-scale models.

One methodological limitation of the present framework is its reliance on block maxima, meaning only the single largest value per year per cell is used, and sub-annual extremes are missed. While this choice is theoretically justified within extreme value theory and aligns naturally with max-stable modelling, future work could explore peaks-over-threshold (POT) extensions in a spatial setting [1] to improve data efficiency while retaining theoretical consistency.

New research directions.. By providing a clean Python API and CLI, **weatherisk** lowers the entry barrier for researchers who wish to apply spatial clustering of extremes to new domains (e.g., wind speed, sea-level pressure) or different datasets (e.g., ERA5, CMIP6 projections).

Education.. The step-by-step pipeline design, the comprehensive test suite, and the documented code examples (Appendix Appendix A) make the package a practical resource for practitioners. A Jupyter notebook tutorial (`notebooks/weatherisk_quickstart.ipynb`) that walks through the full pipeline on synthetic data is included in the repository.

6. Conclusions

We have presented **weatherisk**, a Python package that implements the complete pipeline for spatial clustering of climate extremes via max-stable processes. The package covers the full preprocessing chain—STL detrending, annual block-maxima extraction, GEV fitting, and Fréchet transformation—followed by local composite-likelihood estimation of anisotropy parameters and regionalisation using Local Estimates Clustering (LEC) or Extremal Dependence Clustering (EDC) [4]. This consolidates a previously fragmented R/Shell workflow into a single, tested, pip-installable library.

The software has been verified at three complementary levels: per-function agreement with the historical R implementation of Contzen et al. [4] to $\leq 10^{-10}$ relative error, end-to-end CMIP6 parity on a controlled subset with exact cluster counts, and broader structural and regression testing covering checkpointing, parallelism, and parameter integrity. The full Figure 9 case-study application on the AWI-ESM-1-1-LR T63 grid confirms that **weatherisk** is both methodologically faithful to the reference implementation and scalable to publication-level workloads on HPC infrastructure.

Together, these checks give confidence that the results are correct and reproducible. The spatial clustering condenses a large climate field into a compact description of where extremes co-occur—a result that speaks directly to practical questions in reinsurance, infrastructure planning, and climate services. Two limits of the current design should be stated plainly: the block-maxima approach uses only one value per grid cell per year, so shorter-duration extremes are not captured; and the quadratic cost of pairwise comparison makes kilometre-scale grids impractical with the current algorithm.

Several directions follow naturally from these boundaries. Statistically, peaks-over-threshold extensions in a spatial setting [1] would improve data efficiency while preserving the theoretical framework, and sparse or graph-based dissimilarity approximations would address the quadratic scaling. On

the engineering side, a compiled Rust extension (`weatherisk_core`) built via PyO3 bindings [17] demonstrated a $2.8\times$ end-to-end speed-up on synthetic benchmarks, with the local MLE step as the principal bottleneck; releasing it as a tested, opt-in backend is the most immediate next step. Looking further ahead, GPU parallelisation via CuPy [15] or Numba [16] offers a natural path to handling next-generation kilometre-scale Earth system simulations without changing the underlying statistical framework.

Outlook: Digital Earth System Twins.. High-resolution digital replicas of the Earth system—such as those being developed within the European Destination Earth initiative—produce continuous streams of model output that demand automated analysis tools operating at the same scale and cadence. Spatial clustering of extremes is a natural component of such a pipeline: it converts the full field of local dependence estimates into a discrete set of homogeneous regions that can be monitored over time, compared across ensemble members, and passed to impact and risk models without manual intervention. Extending `weatherisk` towards this setting would require three concrete steps: incremental updating of cluster boundaries as new data arrive rather than batch reprocessing from scratch; direct linkage of region labels to socio-economic exposure databases; and an objective criterion for detecting when a region’s extremal structure has shifted enough to warrant re-clustering. These extensions would bring the current batch-oriented research tool closer to the operational systems that climate services and adaptation planning increasingly require.

`weatherisk` is available under the MIT licence.

Acknowledgements

The authors thank the Alfred Wegener Institute Helmholtz Centre for Polar and Marine Research (AWI) for access to the *albedo* computing cluster, on which the Figure 9 case-study pipeline was executed. We thank J. Contzen for making the original R implementation publicly available and for helpful discussions during the development of `weatherisk`.

Author contributions.. W. Kchikach designed and implemented the software, conducted all numerical experiments, and wrote the first draft of the manuscript. T. Dickhaus and G. Lohmann supervised the work, contributed to the scientific conception, and revised the manuscript. All authors read and approved the final version.

Funding.. W. Kchikach is supported by the MarDATA Helmholtz School for Marine Data Science [grant number — to be confirmed]. T. Dickhaus and G. Lohmann acknowledge support from [funding source — to be confirmed].

Appendix A. Key code examples

This appendix collects the main code snippets that a practitioner needs to run the pipeline flows shown in the paper.

Appendix A.1. A.1 Synthetic validation (Flow 1)

```
from weatherisk.parameters import get_preset
from weatherisk.simulation import simulate_non_stat
from weatherisk.grid import make_grid
from weatherisk. estimation import (
    calc_local_estimates,
    smooth_estimates,
    calc_estimates_in_clusters,
)
from weatherisk.clustering import (
    ellipse_dissimilarity,
    cluster_from_dissimilarity,
    saunders_clustering,
)
from weatherisk.plotting import plot_cluster_map

# 1. Load a parameter preset
p = get_preset("stripes")

# 2. Build a grid and simulate data
grid = make_grid(resolution=10)
data = simulate_non_stat(
    p.params, grid, n_sim=20,
    df=p.df, alpha=p.alpha, seed=42,
)

# 3. Estimate local anisotropy parameters
raw_est = calc_local_estimates(
    data, grid, df=p.df, alpha=p.alpha,
)
est = smooth_estimates(raw_est, grid)

# 4. Cluster by ellipse-overlap dissimilarity
D = ellipse_dissimilarity(est, grid)
clusters = cluster_from_dissimilarity(
    D, quantile_threshold=0.3,
)

# 5. Re-estimate within clusters
in_clust = calc_estimates_in_clusters(
    data, clusters, grid,
    df=p.df, alpha=p.alpha,
)
```



```
# 6. Plot
fig = plot_cluster_map(clusters, grid)
fig.savefig("lec_clusters.pdf")
```

Listing 3: Complete synthetic validation workflow.

Appendix A.2. A.2 CMIP6 Figure 9 *case-study workflow*

```
# On HPC (albedo SLURM cluster):
$ sbatch hpc/run_fig9_clean.slurm

# Or locally (requires CMIP6 data):
$ python scripts/reproduce_fig9.py \
  --data-dir /path/to/AWI-ESM-1-1-LR/pr/ \
  --workers 8

# Programmatic access:
from weatherisk.cmip6_pipeline import (
    run_cmip6_pipeline,
    CMIP6Config,
)
cfg = CMIP6Config() # paper defaults
results = run_cmip6_pipeline(cfg)
# results contains: labels_lec, labels_edc,
# k_lec, k_edc, smoothed, frechet, ...
```

Listing 4: Running the Figure 9 case study on HPC or locally.

References

- [1] A.C. Davison, S.A. Padoan, M. Ribatet, Statistical modeling of spatial extremes, *Stat. Sci.* 27 (2) (2012) 161–186.
- [2] L. de Haan, A. Ferreira, *Extreme Value Theory: An Introduction*, Springer, New York, 2006.
- [3] S.A. Padoan, M. Ribatet, S.A. Sisson, Likelihood-based inference for max-stable processes, *J. Am. Stat. Assoc.* 105 (489) (2010) 263–277.
- [4] J. Contzen, T. Dickhaus, G. Lohmann, Regionalization approaches for the spatial analysis of extremal dependence, *Extremes* 28 (2025) 713–737. <https://doi.org/10.1007/s10687-025-00519-2>
- [5] G. Bocharov, pyextremes: Extreme value analysis in Python, <https://github.com/georgebv/pyextremes>, 2023.

- [6] M. Ribatet, SpatialExtremes: Modelling Spatial Extremes, R package, <https://CRAN.R-project.org/package=SpatialExtremes>, 2023.
- [7] M. Schlather, Models for stationary max-stable random fields, *Extremes* 5 (1) (2002) 33–44.
- [8] K. Saunders, A.G. Stephenson, P.J. Taylor, D. Karoly, The spatial clustering of rainfall extremes and the influence of El Niño–Southern Oscillation, *J. Climate* 34 (12) (2021) 4859–4878.
- [9] D. Cooley, P. Naveau, P. Poncet, Variograms for spatial max-stable random fields, in: P. Bertail, P. Soulier, P. Doukhan (Eds.), *Dependence in Probability and Statistics*, Springer, 2006, pp. 373–390.
- [10] R.B. Cleveland, W.S. Cleveland, J.E. McRae, I. Terpenning, STL: A seasonal-trend decomposition procedure based on loess, *J. Official Stat.* 6 (1) (1990) 3–73.
- [11] G. Lohmann, M. Butzin, N. Eissner, X. Shi, C. Stepanek, Abrupt climate and weather changes across time scales, *Paleoceanogr. Paleoclimatol.* 35 (9) (2020) e2019PA003782. <https://doi.org/10.1029/2019PA003782>
- [12] M. Rocklin, Dask: Parallel computation with blocked algorithms and task scheduling, in: *Proceedings of the 14th Python in Science Conference*, 2015, pp. 130–136.
- [13] J.J. Hamman et al., xarray: N-D labeled arrays and datasets in Python, *J. Open Res. Softw.* 6 (1) (2018). <https://doi.org/10.5334/jors.148>
- [14] R. Abernathey et al., Pangeo: A community platform for big data geoscience, *AGU Adv.* 2 (1) (2021). <https://doi.org/10.1029/2019AV000232>
- [15] R. Okuta, Y. Unno, D. Nishino, S. Hido, C. Loomis, CuPy: A NumPy-compatible library for NVIDIA GPU calculations, in: *Proceedings of Workshop on Machine Learning Systems*, 2017.
- [16] S.K. Lam, A. Pitrou, S. Seibert, Numba: A LLVM-based Python JIT compiler, in: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6.
- [17] PyO3 Contributors, PyO3: Rust bindings for the Python interpreter, <https://github.com/PyO3/pyo3>, 2023.